

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Mounted at /content/drive

```
1 import zipfile
```

```
1 zip_path = "/content/drive/MyDrive/task_vii_dataset.zip"
2 extract_path = "/content/dataset"
3
4 with zipfile.ZipFile(zip_path, 'r') as zip_ref:
5     zip_ref.extractall(extract_path)
6
7 print(" extracted !")
```

extracted !

```
1 import os
2
3 for root, dirs, files in os.walk("/content/dataset"):
4     print(root, "->", len(files))
```

```
/content/dataset -> 0
/content/dataset/dataset -> 1
/content/dataset/dataset/val -> 1
/content/dataset/dataset/val/sphere -> 2500
/content/dataset/dataset/val/vort -> 2500
/content/dataset/dataset/val/no -> 2500
/content/dataset/dataset/train -> 1
/content/dataset/dataset/train/sphere -> 10000
/content/dataset/dataset/train/vort -> 10000
/content/dataset/dataset/train/no -> 10000
/content/dataset/__MACOSX -> 0
/content/dataset/__MACOSX/dataset -> 1
/content/dataset/__MACOSX/dataset/val -> 1
/content/dataset/__MACOSX/dataset/train -> 1
```

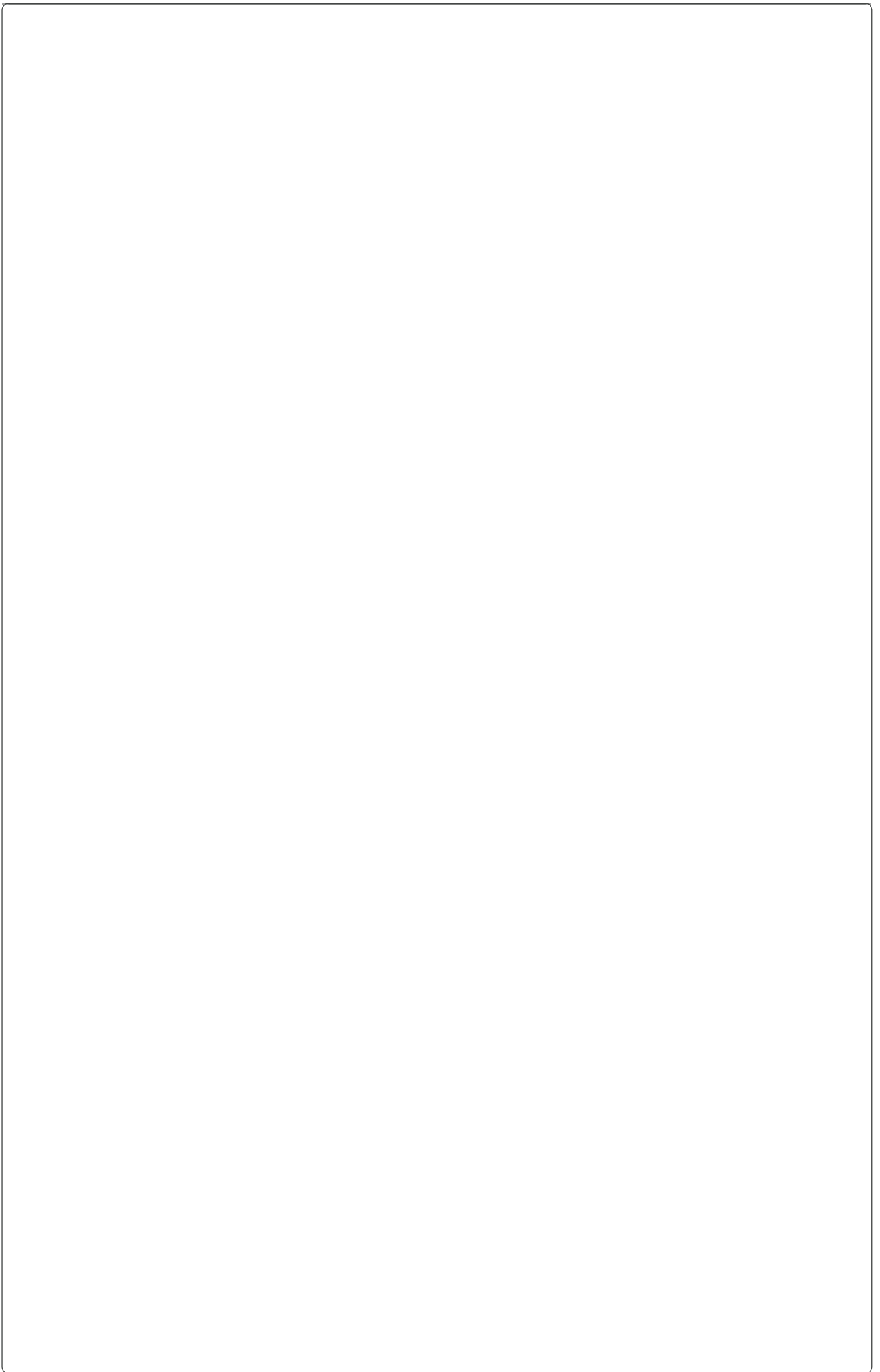
```
1 import os
2 import glob
3 import random
4 import numpy as np
5
6 import torch
7 import torch.nn as nn
8 import torch.nn.functional as F
9 import torchvision.models as models
10 import torchvision.transforms as T
11
12 from torch.utils.data import Dataset, DataLoader
13 from sklearn.model_selection import train_test_split
14 from sklearn.metrics import roc_auc_score, accuracy_score
15 from sklearn.metrics import roc_curve, auc
16 from sklearn.preprocessing import label_binarize
17
18 import matplotlib.pyplot as plt
```

```
1 dataset_base_path = '/content/dataset/dataset'
2 subdirs = ['train', 'val']
3 categories = ['no', 'sphere', 'vort']
4
5 print(f"Dataset Base Path: {dataset_base_path}")
6 print(f"Subdirectories: {subdirs}")
7 print(f"Categories: {categories}")
```

Dataset Base Path: /content/dataset/dataset
Subdirectories: ['train', 'val']
Categories: ['no', 'sphere', 'vort']

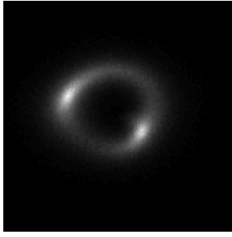
```
1 import matplotlib.pyplot as plt
2 import os
3 import random
4 import numpy as np
5
6 num_images_to_display = 4
7
8 for subdir in subdirs:
9     for category in categories:
10         current_path = os.path.join(dataset_base_path, subdir, category)
11
```

```
12     print(f"\nVisualizing {num_images_to_display} random images from: {subdir}/{category}")
13
14     image_files = [
15         os.path.join(current_path, f)
16         for f in os.listdir(current_path)
17         if f.lower().endswith('.numpy') and not f.startswith('.')
18     ]
19
20     if not image_files:
21         print(f"No valid .numpy files found in {current_path} after filtering. Skipping.")
22         continue
23
24     selected_images = random.sample(image_files, min(num_images_to_display, len(image_files)))
25
26     plt.figure(figsize=(15, 3))
27     for i, img_path in enumerate(selected_images):
28         plt.subplot(1, len(selected_images), i + 1)
29         img_array = np.load(img_path)
30
31         if img_array.ndim == 3 and img_array.shape[0] == 1:
32             img_array = img_array.squeeze(0)
33
34         plt.imshow(img_array, cmap='gray')
35         plt.title(f"{category} ({os.path.basename(img_path)})")
36         plt.axis('off')
37     plt.tight_layout()
38     plt.show()
39
```

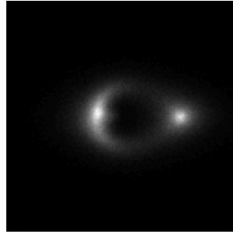


Visualizing 4 random images from: train/no

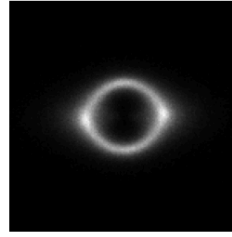
no (4784.npy)



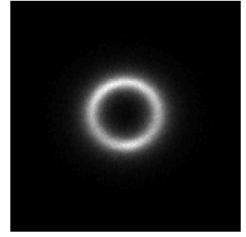
no (8225.npy)



no (2591.npy)

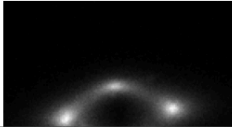


no (8260.npy)



Visualizing 4 random images from: train/sphere

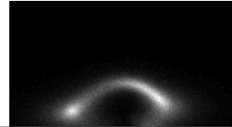
sphere (1456.npy)



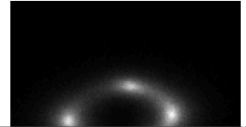
sphere (8839.npy)



sphere (2732.npy)



sphere (6888.npy)



```
1 dataset_base_path = "/content/dataset/dataset"
2
3 categories = ["no", "sphere", "vort"]
4 label_map = {"no":0, "sphere":1, "vort":2}
```

```
1 train_transforms = T.Compose([
2     T.ToPILImage(),
3     T.RandomHorizontalFlip(),
4     T.RandomVerticalFlip(),
5     T.RandomRotation(20),
6     T.RandomResizedCrop(224, scale=(0.8,1.0)),
7     T.ToTensor(),
8     T.Normalize(
9         mean=[0.485,0.456,0.406],
10        std=[0.229,0.224,0.225]
11    )
12 ])
13
14 val_transforms = T.Compose([
15     T.ToPILImage(),
16     T.Resize((224,224)),
17     T.ToTensor(),
18     T.Normalize(
19         mean=[0.485,0.456,0.406],
20         std=[0.229,0.224,0.225]
21     )
22 ])
```

```
1 class LensDataset(Dataset):
2
3     def __init__(self, files, labels, transform=None):
4
5         self.files = files
6         self.labels = labels
7         self.transform = transform
8
9     def __len__(self):
10         return len(self.files)
11
12     def __getitem__(self, idx):
13
14         img = np.load(self.files[idx]).astype(np.float32)
15
16         if img.ndim == 3 and img.shape[0] == 1:
17             img = img.squeeze(0)
18
19         img = (img - img.min())/(img.max()-img.min()+1e-6)
20
21         img = np.stack([img,img,img],axis=-1)
22
23         if self.transform:
24             img = self.transform(img)
25
26         label = torch.tensor(self.labels[idx]).long()
27
28         return img,label
```

```
1 all_files = []
2 all_labels = []
```

```

3
4 for subdir in ["train","val"]:
5     for category in categories:
6
7         path = os.path.join(dataset_base_path,subdir,category)
8
9         files = glob.glob(path+"/*.npz")
10        files = [f for f in files if not os.path.basename(f).startswith("_")]
11
12        labels = [label_map[category]]*len(files)
13
14        all_files.extend(files)
15        all_labels.extend(labels)
16
17 print("Total images:",len(all_files))

```

Total images: 37500

```

1 train_files,val_files,train_labels,val_labels = train_test_split(
2     all_files,
3     all_labels,
4     test_size=0.1,
5     stratify=all_labels,
6     random_state=42
7 )

```

```

1 len(train_files)/len(all_files) * 100

```

90.0

```

1 train_dataset = LensDataset(train_files,train_labels,train_transforms)
2 val_dataset = LensDataset(val_files,val_labels,val_transforms)
3
4 train_loader = DataLoader(
5     train_dataset,
6     batch_size=64,
7     shuffle=True,
8     num_workers=2,
9     pin_memory=True
10 )
11
12 val_loader = DataLoader(
13     val_dataset,
14     batch_size=64,
15     num_workers=2,
16     pin_memory=True
17 )

```

EfficientNet Model Results

```

1 class EfficientNetPINN(nn.Module):
2
3     def __init__(self):
4
5         super().__init__()
6
7         base = models.efficientnet_b0(
8             weights=models.EfficientNet_B0_Weights.IMAGENET1K_V1
9         )
10
11         self.encoder = base.features
12
13         self.pool = nn.AdaptiveAvgPool2d(1)
14
15         self.classifier = nn.Linear(1280,3)
16
17         self.psi_head = nn.Sequential(
18             nn.Conv2d(1280,256,3,padding=1),
19             nn.ReLU(),
20             nn.Conv2d(256,64,3,padding=1),
21             nn.ReLU(),
22             nn.Conv2d(64,1,1),
23             nn.Upsample(size=(224, 224), mode='bilinear', align_corners=False) # Upsample to original image size
24         )
25
26     def forward(self,x):
27
28         feat = self.encoder(x)

```

```

29
30     psi = self.psi_head(feats)
31
32     pooled = self.pool(feats).flatten(1)
33
34     logits = self.classifier(pooled)
35
36     return logits,psi

```

```

1 def laplacian(x):
2
3     dx = torch.gradient(x,dim=2)[0]
4     dy = torch.gradient(x,dim=3)[0]
5
6     dxx = torch.gradient(dx,dim=2)[0]
7     dyy = torch.gradient(dy,dim=3)[0]
8
9     return dxx + dyy

```

```

1 def poisson_loss(psi,img):
2
3     lap = laplacian(psi)
4
5     kappa = img.mean(dim=1,keepdim=True)
6
7     residual = lap - 2*kappa
8
9     return torch.mean(residual**2)
10
11
12 def smoothness_loss(psi):
13
14     gx = torch.gradient(psi,dim=2)[0]
15     gy = torch.gradient(psi,dim=3)[0]
16
17     return torch.mean(gx**2 + gy**2)

```

```

1 criterion = nn.CrossEntropyLoss()
2
3 lambda_poisson = 0.001
4 lambda_smooth = 0.0005

```

```

1 def train_epoch_pinn(model,loader,optimizer,device):
2
3     model.train()
4
5     total_loss=0
6
7     for x,y in loader:
8
9         x=x.to(device)
10        y=y.to(device)
11
12        optimizer.zero_grad()
13
14        logits,psi = model(x)
15
16        cls = criterion(logits,y)
17
18        phys = poisson_loss(psi,x)
19        smooth = smoothness_loss(psi)
20
21        loss = cls + lambda_poisson*phys + lambda_smooth*smooth
22
23        loss.backward()
24
25        torch.nn.utils.clip_grad_norm_(model.parameters(),1)
26
27        optimizer.step()
28
29        total_loss += loss.item()
30
31    return total_loss/len(loader)

```

```

1 def evaluate_model(model,loader,device):
2
3     model.eval()
4
5     all_probs=[]

```

```

6     all_preds=[]
7     all_labels=[]
8
9     with torch.no_grad():
10
11         for x,y in loader:
12
13             x=x.to(device)
14
15             if isinstance(model,EfficientNetPINN):
16                 logits,_=model(x)
17             else:
18                 logits=model(x)
19
20             probs = F.softmax(logits,1).cpu()
21
22             preds = torch.argmax(probs,1)
23
24             all_probs.append(probs)
25             all_preds.append(preds)
26             all_labels.append(y)
27
28     all_probs=torch.cat(all_probs).numpy()
29     all_preds=torch.cat(all_preds).numpy()
30     all_labels=torch.cat(all_labels).numpy()
31
32     auc=roc_auc_score(all_labels,all_probs,multi_class="ovr")
33     acc=accuracy_score(all_labels,all_preds)
34
35     return auc,acc

```

```

1 def train_model(model,train_loader,val_loader,epochs=15,name="model"):
2
3     device = "cuda" if torch.cuda.is_available() else "cpu"
4
5     model=model.to(device)
6
7     optimizer = torch.optim.AdamW(
8         model.parameters(),
9         lr=3e-4,
10        weight_decay=1e-4
11    )
12
13    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
14        optimizer,
15        T_max=epochs
16    )
17
18    history={"train_loss":[],"val_auc":[],"val_acc":[]}
19
20    best_auc=0
21
22    for epoch in range(epochs):
23
24        train_loss = train_epoch_pinn(model,train_loader,optimizer,device)
25        val_auc,val_acc = evaluate_model(model,val_loader,device)
26
27        scheduler.step()
28
29        history["train_loss"].append(train_loss)
30        history["val_auc"].append(val_auc)
31        history["val_acc"].append(val_acc)
32
33        print(
34            f"{name} | Epoch {epoch+1}/{epochs}"
35            f" | Loss {train_loss:.4f}"
36            f" | AUC {val_auc:.4f}"
37            f" | Acc {val_acc:.4f}"
38        )
39
40        if val_auc>best_auc:
41
42            best_auc=val_auc
43
44            torch.save(model.state_dict(),f"{name}_best.pth")
45
46    return history

```

```

1 pinn = EfficientNetPINN()
2
3 history_pinn = train_model(

```

```

4     pinn,
5     train_loader,
6     val_loader,
7     name="efficientnet_pinn"
8 )

```

efficientnet_pinn	Epoch 1/15	Loss 0.9746	AUC 0.8517	Acc 0.6592
efficientnet_pinn	Epoch 2/15	Loss 0.6289	AUC 0.9274	Acc 0.7848
efficientnet_pinn	Epoch 3/15	Loss 0.4821	AUC 0.9532	Acc 0.8371
efficientnet_pinn	Epoch 4/15	Loss 0.4091	AUC 0.9644	Acc 0.8520
efficientnet_pinn	Epoch 5/15	Loss 0.3572	AUC 0.9672	Acc 0.8480
efficientnet_pinn	Epoch 6/15	Loss 0.3241	AUC 0.9760	Acc 0.8851
efficientnet_pinn	Epoch 7/15	Loss 0.2956	AUC 0.9765	Acc 0.8917
efficientnet_pinn	Epoch 8/15	Loss 0.2703	AUC 0.9842	Acc 0.9091
efficientnet_pinn	Epoch 9/15	Loss 0.2527	AUC 0.9831	Acc 0.9115
efficientnet_pinn	Epoch 10/15	Loss 0.2385	AUC 0.9856	Acc 0.9240
efficientnet_pinn	Epoch 11/15	Loss 0.2187	AUC 0.9876	Acc 0.9251
efficientnet_pinn	Epoch 12/15	Loss 0.2065	AUC 0.9889	Acc 0.9312
efficientnet_pinn	Epoch 13/15	Loss 0.1980	AUC 0.9891	Acc 0.9352
efficientnet_pinn	Epoch 14/15	Loss 0.1885	AUC 0.9897	Acc 0.9352
efficientnet_pinn	Epoch 15/15	Loss 0.1944	AUC 0.9900	Acc 0.9379

```

1 def plot_history(history):
2
3     plt.figure(figsize=(15,4))
4
5     plt.subplot(1,3,1)
6     plt.plot(history["train_loss"])
7     plt.title("Train Loss")
8
9     plt.subplot(1,3,2)
10    plt.plot(history["val_auc"])
11    plt.title("Validation AUC")
12
13    plt.subplot(1,3,3)
14    plt.plot(history["val_acc"])
15    plt.title("Validation Accuracy")
16
17    plt.show()

```

```

1 def plot_roc(model,loader,device):
2
3     model.eval()
4
5     probs=[]
6     labels=[]
7
8     with torch.no_grad():
9
10        for x,y in loader:
11
12            x=x.to(device)
13
14            if isinstance(model,EfficientNetPINN):
15                logits,_=model(x)
16            else:
17                logits=model(x)
18
19            p=F.softmax(logits,1).cpu()
20
21            probs.append(p)
22            labels.append(y)
23
24        probs=torch.cat(probs).numpy()
25        labels=torch.cat(labels).numpy()
26
27        labels_bin=label_binarize(labels,classes=[0,1,2])
28
29        plt.figure()
30
31        print("AUC per class:")
32
33        for i,name in enumerate(["no","sphere","vort"]):
34
35            fpr,tpr,_=roc_curve(labels_bin[:,i],probs[:,i])
36            class_auc = auc(fpr,tpr)
37
38            print(f"{name}: {class_auc:.4f}")
39
40            plt.plot(fpr,tpr,label=f"{name} AUC={class_auc:.2f}")
41
42        plt.plot([0,1],[0,1], 'k--')

```

```

43
44 plt.legend()
45
46 plt.title("ROC Curve")
47
48 plt.show()

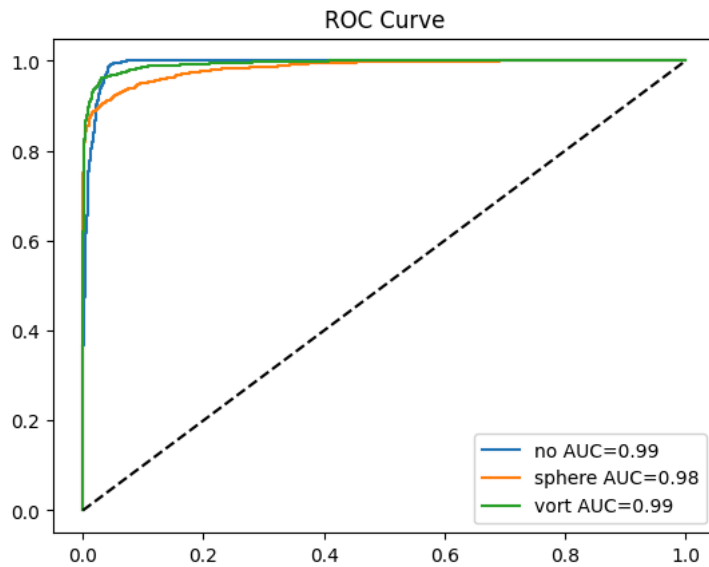
```

```

1 device="cuda" if torch.cuda.is_available() else "cpu"
2
3 plot_roc(pinn, val_loader, device)

```

AUC per class:
no: 0.9924
sphere: 0.9842
vort: 0.9936



```

1 macro_auc, _ = evaluate_model(pinn, val_loader, device)
2 print(f"Macro AUC: {macro_auc:.4f}")

```

Macro AUC: 0.9900

DenseNet Model Results

```

1 class DenseNetPINN(nn.Module):
2
3     def __init__(self):
4
5         super().__init__()
6
7         base = models.densenet121(
8             weights=models.DenseNet121_Weights.IMAGENET1K_V1
9         )
10
11         self.encoder = base.features
12
13         self.pool = nn.AdaptiveAvgPool2d(1)
14
15         self.classifier = nn.Linear(1024, 3)
16
17         self.psi_head = nn.Sequential(
18
19             nn.Conv2d(1024, 256, 3, padding=1),
20             nn.ReLU(),
21
22             nn.Conv2d(256, 64, 3, padding=1),
23             nn.ReLU(),
24
25             nn.Conv2d(64, 1, 1),
26
27             nn.Upsample(
28                 size=(224, 224),
29                 mode="bilinear",
30                 align_corners=False
31             )
32         )
33

```

```

34     def forward(self,x):
35
36         feat = self.encoder(x)
37
38         psi = self.psi_head(feat)
39
40         pooled = self.pool(feat).flatten(1)
41
42         logits = self.classifier(pooled)
43
44         return logits,psi

```

```

1 train_dataset = LensDataset(train_files,train_labels,train_transforms)
2 val_dataset = LensDataset(val_files,val_labels,val_transforms)
3
4 train_loader = DataLoader(
5     train_dataset,
6     batch_size=64,
7     shuffle=True,
8     num_workers=2,
9     pin_memory=True
10 )
11
12 val_loader = DataLoader(
13     val_dataset,
14     batch_size=64,
15     num_workers=2,
16     pin_memory=True
17 )

```

```

1 def evaluate_model(model,loader,device):
2
3     model.eval()
4
5     all_probs=[]
6     all_preds=[]
7     all_labels=[]
8
9     with torch.no_grad():
10
11         for x,y in loader:
12
13             x=x.to(device)
14
15             output = model(x)
16
17             if isinstance(output, tuple):
18                 logits = output[0]
19             else:
20                 logits = output
21
22             probs = F.softmax(logits,1).cpu()
23
24             preds = torch.argmax(probs,1)
25
26             all_probs.append(probs)
27             all_preds.append(preds)
28             all_labels.append(y)
29
30     all_probs=torch.cat(all_probs).numpy()
31     all_preds=torch.cat(all_preds).numpy()
32     all_labels=torch.cat(all_labels).numpy()
33
34     auc=roc_auc_score(all_labels,all_probs,multi_class="ovr")
35     acc=accuracy_score(all_labels,all_preds)
36
37     return auc,acc

```

```

1 pinn = DenseNetPINN()
2
3 history_pinn = train_model(
4     pinn,
5     train_loader,
6     val_loader,
7     name="densenet_pinn"
8 )

```

densenet_pinn	Epoch 1/15	Loss 0.7597	AUC 0.9484	Acc 0.8283
densenet_pinn	Epoch 2/15	Loss 0.4192	AUC 0.9731	Acc 0.8829
densenet_pinn	Epoch 3/15	Loss 0.3563	AUC 0.9760	Acc 0.8805
densenet_pinn	Epoch 4/15	Loss 0.3148	AUC 0.9812	Acc 0.8976

densenet_pinn	Epoch 5/15	Loss 0.2882	AUC 0.9838	Acc 0.9072
densenet_pinn	Epoch 6/15	Loss 0.2626	AUC 0.9895	Acc 0.9333
densenet_pinn	Epoch 7/15	Loss 0.2378	AUC 0.9897	Acc 0.9387
densenet_pinn	Epoch 8/15	Loss 0.2216	AUC 0.9847	Acc 0.9211
densenet_pinn	Epoch 9/15	Loss 0.1991	AUC 0.9932	Acc 0.9515
densenet_pinn	Epoch 10/15	Loss 0.1800	AUC 0.9925	Acc 0.9499
densenet_pinn	Epoch 11/15	Loss 0.1663	AUC 0.9944	Acc 0.9539
densenet_pinn	Epoch 12/15	Loss 0.1548	AUC 0.9950	Acc 0.9621
densenet_pinn	Epoch 13/15	Loss 0.1431	AUC 0.9954	Acc 0.9640
densenet_pinn	Epoch 14/15	Loss 0.1314	AUC 0.9955	Acc 0.9637
densenet_pinn	Epoch 15/15	Loss 0.1253	AUC 0.9957	Acc 0.9653

```

1 def plot_roc(model,loader,device):
2
3     model.eval()
4
5     probs=[]
6     labels=[]
7
8     with torch.no_grad():
9
10        for x,y in loader:
11
12            x=x.to(device)
13
14            output = model(x)
15
16            if isinstance(output, tuple):
17                logits = output[0]
18            else:
19                logits = output
20
21            p=F.softmax(logits,1).cpu()
22
23            probs.append(p)
24            labels.append(y)
25
26        probs=torch.cat(probs).numpy()
27        labels=torch.cat(labels).numpy()
28
29        labels_bin=label_binarize(labels,classes=[0,1,2])
30
31        plt.figure()
32
33        print("AUC per class:")
34
35        for i,name in enumerate(["no","sphere","vort"]):
36
37            fpr,tpr,_=roc_curve(labels_bin[:,i],probs[:,i])
38            class_auc = auc(fpr,tpr)
39
40            print(f"{name}: {class_auc:.4f}")
41
42            plt.plot(fpr,tpr,label=f"{name} AUC={class_auc:.4f}")
43
44        plt.plot([0,1],[0,1], 'k--')
45
46        plt.legend()
47
48        plt.title("ROC Curve")
49
50        plt.show()

```

```

1 device="cuda" if torch.cuda.is_available() else "cpu"
2
3 plot_roc(pinn,val_loader,device)

```